Informe Caso 3 TIC

Nombre-Código-Login:

Fabio Andrés Salazar Ochoa-202511238-fa.salazar

Juan David Gómez Melo-202510127-jd.gomezm1

Funcionamiento global del Sistema.

El sistema corre en el main (Main.java), donde se lee el archivo de configuración y, a partir de sus parámetros, se inicializa el sistema. Primero se crean los objetos compartidos: el buzón de entrada (BuzonEntrada.java), la zona de espera (ZonaEspera.java) y los tres controladores (ControladorProcesadores.java, ControladorMemoria.java y ControladorAlmacenamiento.java). Luego se crean y se inician los threads usuarios (Usuario.java), uno por cada usuario según el parámetro nu, y después los threads gestores (Gestor.java), uno por cada gestor según el parámetro ng.

Una vez iniciados los threads, cada usuario genera base*id solicitudes, donde los ids van de 1 hasta el número de usuarios (nu). Las solicitudes se crean con valores aleatorios y se depositan en el buzón. Si el buzón está lleno, el usuario entra en espera pasiva hasta que se le notifique que hay espacio.

Por otro lado, cada gestor retira solicitudes del buzón mientras queden usuarios activos o solicitudes en el buzón (eso revisa el método terminado() del buzón). Antes de retirar revisa la zona de espera: si alcanzó el umbral, espera a que se vacíe. Si el buzón está vacío, hace Thread.yield() y vuelve a intentar. Con cada solicitud crea un thread de máquina virtual (MaquinaVirtual.java) y lo inicia.

Una vez iniciadas, cada MV intenta reservar los 3 recursos (Procesadores, Memoria, Almacenamiento) en ese orden en específico, si alguno falla, devuelve todos los recursos que haya alcanzado reservar y entra a la zona hasta que se le notifique que puede volver a intentar reservar, y realiza este proceso hasta que lo logre, una vez tenga sus recursos reservados se inicializa y corre por un rango de tiempo (4000 a 40000ms), cuando termina libera todos los recursos y avisa.

El programa termina cuando todos los Threads terminan, esto se logra con los join() usados en main, y se verifica que el estado de todo el sistema sea coherente con su estado de terminal, y una vez todo esto suceda, se termina el programa.

Sincronización entre objetos.

- *Usuario-BuzonEntrada.*
  Aquí el usuario cumple la función de productor del patrón productores-consumidores y el buzón cumple la función de buffer. El estado compartido es la cola y el contador

usuariosActivos. depositar() es synchronized para evitar condiciones de carrera sobre la cola. Si está llena, el usuario hace wait con una espera pasiva y al despertar vuelve a revisar la condición con un while. Los usuarios se despiertan con el notifyAll de retirar(), que ocurre cada vez que se libera un espacio. usuarioTermino() es synchronized para evitar condiciones de carrera sobre usuariosActivos.

- *Gestor-BuzonEntrada.*
  Aquí el gestor cumple la función de consumidor del patrón productores-consumidores y el buzón cumple la función de buffer. El estado compartido es la cola y el contador de usuarios. retirar() es synchronized, lo que garantiza que cada solicitud la reciba un único gestor aunque varios compitan. No bloquea con wait: si el buzón está vacío devuelve null y el gestor hace espera semi-activa con Thread.yield(). terminado() es synchronized para evitar inconsistencias en las cuentas y revisa en una sola sección crítica que no hay usuarios activos y que la cola está vacía. Si se revisaran por separado habría una condición de carrera: el usuario podría depositar y terminar entre las dos lecturas, y el gestor saldría dejando una solicitud que nunca se procesara.
- *Gestor-ZonaEspera.*
  El estado compartido es el contador pendientes. alcanzaUmbral() es synchronized para leer un valor consistente. La espera total se implementa con wait dentro de un while (pendientes > 0) en esperarVaciado(), y el notifyAll lo ejecuta salir() cuando el contador llega a cero. La condición se vuelve a evaluar al despertar porque otro hilo pudo modificar el contador mientras tanto. Como el gestor revisa el umbral antes de retirar del buzón, mientras espera no consume solicitudes, así que las pendientes no compiten con solicitudes nuevas.
- *Gestor-MaquinaVirtual.*
  El gestor crea la MV con su solicitud, la zona y los controladores, y la inicia con start(). start() garantiza que la MV ve todo lo que el gestor preparó antes de iniciarla. Después, el gestor y la MV no comparten estado que cambie: la solicitud es inmutable, los atributos de la MV son final, y la zona y los controladores tienen su propio monitor. Por eso este par no necesita un monitor propio. La lista maquinasVirtuales sí se comparte entre todos los gestores, que la escriben al tiempo, así que cada gestor agrega dentro de synchronized (maquinasVirtuales). Main la recorre solo después de hacer join() a todos los gestores. En ese momento ya nadie la modifica y join() garantiza que todo lo agregado es visible. El gestor no espera a las MVs: esa sincronización de terminación la hace Main con join() sobre cada MV.
- *MaquinaVirtual-ZonaEspera.*
  El estado compartido es generacion y pendientes, ambos con métodos synchronized para evitar condición de carrera en sus cuentas. La espera pasiva usa wait sobre el monitor de la zona, y el problema principal es el aviso perdido, pues si una MV falla al reservar y otra libera justo antes de que la primera ejecute wait, el notifyAll se pierde y la MV quedaría dormida para siempre. Para evitar esto, la MV lee la generación antes de intentar reservar y dormir(gen) solo hace wait mientras generacion == gen; como cada liberación incrementa

la generación, una liberación previa impide el bloqueo. Se usa notifyAll y no notify porque las MVs esperan condiciones distintas (cantidades distintas): con notify podría despertarse una MV que no puede avanzar mientras otra que sí puede sigue dormida. entrar() se llama solo en el primer fallo y salir() al lograr reservar, de modo que pendientes no llegue a cero mientras hay MVs reintentando, lo que haría que el gestor ingresara solicitudes antes de tiempo.

- *MaquinaVirtual-Controladores de recursos.*
  El estado compartido es el arreglo de propietarios y el contador de unidades libres de cada controlador. reservar() y liberar() son synchronized, y reservar() es de todo o nada para que nunca queden asignaciones parciales. Para prevenir el interbloqueo se rompen dos de las condiciones necesarias, por un lado la espera circular se trata con el orden fijo de reserva (procesadores, memoria, almacenamiento) para todas las MVs, y retener y esperar, devolviendo inmediatamente lo reservado cuando un recurso no está disponible. Cada controlador tiene su propio monitor y la MV nunca toma un monitor mientras sostiene otro, por lo que tampoco hay bloqueo entre locks. Por último, la MV libera los recursos antes de llamar a notificarLiberacion(), para que las MVs despertadas encuentren los recursos ya disponibles y no vuelvan a fallar.
- *Main-Usuarios, Gestores y MVs.*
  Main se sincroniza con los demás hilos mediante join(), que bloquea al hilo principal hasta que el hilo objetivo termine y garantiza la visibilidad de todo lo que este escribió. Espera primero a usuarios y gestores, y luego a las MVs de la lista compartida maquinasVirtuales. Solo después revisa el estado final, porque hacerlo antes sería una condición de carrera con los hilos que aún están en ejecución.

Preguntas del enunciado.

- Si el número de MVs en espera es igual o superior al umbral, el gestor debe esperar a que todas las solicitudes pendientes se procesen (no permite el ingreso de solicitudes adicionales hasta terminar con las pendientes). ¿Qué problema resuelve esta manera de procesar solicitudes?

  Esta manera de procesar solicitudes resuelve la inanición de solicitudes, pues asegura que las solicitudes pendientes no se queden esperando infinitamente por el ingreso constante de nuevas solicitudes.

- Una máquina reserva los recursos que necesita -uno a uno- y siempre en el orden establecido: procesador, memoria y disco. ¿Por qué es importante que todas sigan el mismo orden de solicitud?

  Es importante para evitar una cadena de dependencias cíclicas y bloqueos circulares en las MVs, pues si no existiera el orden predeterminado, una máquina podría estar esperando un tipo de recurso que está siendo reservado por otra que a su vez esta esperando el recurso que la primera maquina esta reservando, por lo que se quedan esperando infinitamente, bloqueándose.

- ¿por qué es importante liberar los recursos asignados cuando no se logra completar la reserva?

  Porque así se evita un interbloqueo, al evitar que los recursos sean reservados cuando en verdad no se pueden utilizar, también permite que por esta misma razón, MVs que en caso de que no se liberaran los recursos, no alcanzan a reservarlos (Por ejemplo una MV que necesita 3 procesadores, y solo hay 2 disponibles pero hay una Máquina que esta reservando innecesariamente 1, si lo liberara la MV que necesita 3 procesadores ya podría acceder)

- Cuando termine su ejecución, la máquina deberá liberar todos los recursos que tenía asignados y despertar otra máquina para que use los recursos. ¿debería despertar una sola o todas? ¿por qué?
  Deben despertarse todas, pues las MVs en espera piden cantidades distintas de recursos y quien libera no sabe cuál de ellas cabe con lo recién liberado, por lo que si despierta una sola y justo la que despertó no pudiera completar su reserva, volvería a dormir sin que nadie más fuera despertado y se interrumpe el flujo del sistema.

Explicación espera total.

Antes de sacar una solicitud del buzón, el gestor revisa si el número de MVs pendientes en la zona de espera llegó al umbral (tam2). Una MV es pendiente desde que falla por primera vez al reservar hasta que consigue los tres recursos.

Si llegó al umbral, el gestor llama a esperarVaciado() y queda en espera pasiva (wait()) mientras haya MVs pendientes. Durante ese tiempo no saca solicitudes del buzón ni crea MVs nuevas.

Las MVs que terminan liberan sus recursos y despiertan a las que esperan, que vuelven a intentar. Cuando la última pendiente consigue sus recursos, el contador queda en 0 y se hace notifyAll(). Eso despierta a los gestores, que siguen sacando solicitudes.

Validación.

Como validación se crearon seis .txt en la carpeta .../casos donde se pretende analizar múltiples situaciones para comprobar el consistente funcionamiento de nuestra implementación del caso. A continuación se listan dichos .txt:

Caso 1: base (caso_base.txt)
4 usuarios, base 2, 2 gestores, buzón 20, umbral 3, 8 procesadores, 16 GiB (páginas de 4 KiB), 64 GiB (bloques de 1024 KiB).

Este caso se creó para analizar la situación en la que el sistema funciona con valores normales: que cada usuario genere la cantidad correcta de solicitudes, que los valores respeten los límites del enunciado y que el programa termine bien. Al correrlo, notamos que los usuarios generaron 2, 4, 6 y 8 solicitudes (20 en total), con procesadores entre 1 y 6, memoria en potencias de 2 hasta 8 GiB y disco par hasta 48 GiB. Varias MVs no consiguieron recursos al principio, esperaron y lo lograron después, incluso la 1-3, que pide casi todo el sistema. Los gestores alcanzaron el umbral dos veces y se detuvieron hasta que la zona se vació. Se cumplieron las 6 condiciones finales.

Caso 2: buzón lleno (caso_buzon.txt)
3 usuarios, base 2, 2 gestores, buzón 1, umbral 3, mismos recursos que el caso base.

Este caso se creó para analizar la situación en la que el buzón se llena todo el tiempo y los usuarios deben esperar para depositar. Al correrlo, notamos que los usuarios esperaron muchas veces con el buzón lleno. Cada vez que un gestor retiraba una solicitud se despertaban y, si otro usuario ya había ocupado el lugar, volvían a esperar. El buzón nunca tuvo más de una solicitud, y mientras los gestores estaban en espera total los usuarios siguieron dormidos. Se crearon las 12 MVs y se cumplieron las 6 condiciones.

Caso 3: umbral (caso_umbral.txt)
4 usuarios, base 1, 2 gestores, buzón 10, umbral 1, 4 procesadores, 8 GiB, 16 GiB.

Este caso se creó para analizar la situación en la que basta con que una sola MV falle para que los gestores entren en espera total. Al correrlo, notamos que los gestores se detuvieron dos veces y, mientras esperaban, no se retiró ninguna solicitud del buzón. Siguieron justo después de que la última MV pendiente consiguió sus recursos. Se crearon las 10 MVs y se cumplieron las 6 condiciones.

Caso 4: concurrencia (caso_concurrencia.txt)
5 usuarios, base 1, 3 gestores, buzón 5, umbral 5, 16 procesadores, 32 GiB, 128 GiB.

Este caso se creó para analizar la situación en la que hay muchos threads trabajando al mismo tiempo (5 usuarios, 3 gestores y 15 MVs), para buscar condiciones de carrera o bloqueos. Al correrlo, notamos que varias MVs se ejecutaron a la vez y muchas esperaron y reintentaron sin problema. Los tres gestores alcanzaron el umbral juntos dos veces y reanudaron juntos. No hubo bloqueos: se crearon las 15 MVs y se cumplieron las 6 condiciones.

Caso 5: tamaño de página (caso_pagina.txt)
3 usuarios, base 1, 2 gestores, buzón 3, umbral 3, 8 procesadores, 8 GiB con páginas de 3 KiB, 16 GiB con bloques de 3 KiB.

Este caso se creó para analizar la situación en la que los GiB no se dividen exacto en páginas o bloques, y los controladores deben redondear. Al correrlo, notamos que las MVs reservaron, esperaron y liberaron normalmente, y que al final la memoria y el almacenamiento quedaron completamente libres. Es decir, se reserva y se libera la misma cantidad de unidades. También vimos que los gestores terminaron antes que las MVs, y que Main esperó a todas antes de validar. Se cumplieron las 6 condiciones.

Caso 6: un solo gestor (caso_un_gestor.txt)
3 usuarios, base 1, 1 gestor, buzón 2, umbral 2, 4 procesadores, 8 GiB, 16 GiB.

Este caso se creó para analizar la situación en la que solo hay un gestor procesando todas las solicitudes. Al correrlo, notamos que ese gestor procesó todo sin problema: alcanzó el umbral, esperó a que la zona se vaciara y siguió. Los usuarios esperaron cuando el buzón se llenó, y el gestor terminó cuando ya no quedaban usuarios ni solicitudes. Se crearon las 6 MVs y se cumplieron las 6 condiciones.